ACTIVIDAD 3

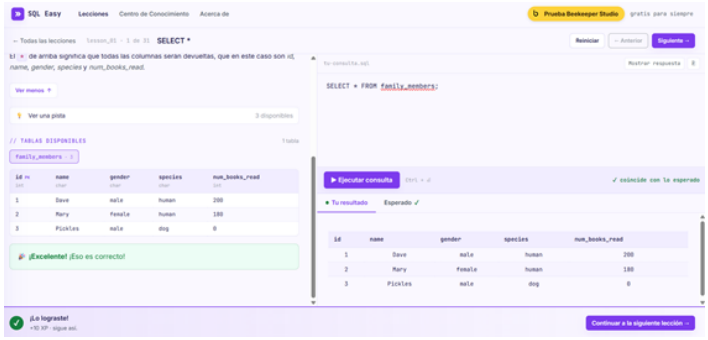
SQL Easy en español

Documentación de 31 consultas: resultados y aplicación en el mundo real

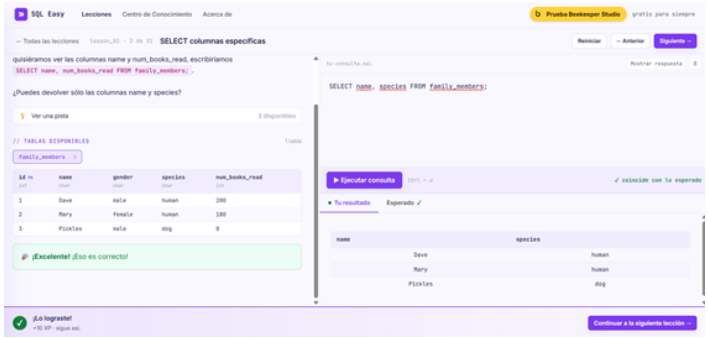
Nombre: Jessica Payano
Fecha: 24 de octubre de 2026
Plataforma: www.sql-easy.com/es

Fichas de ejercicios

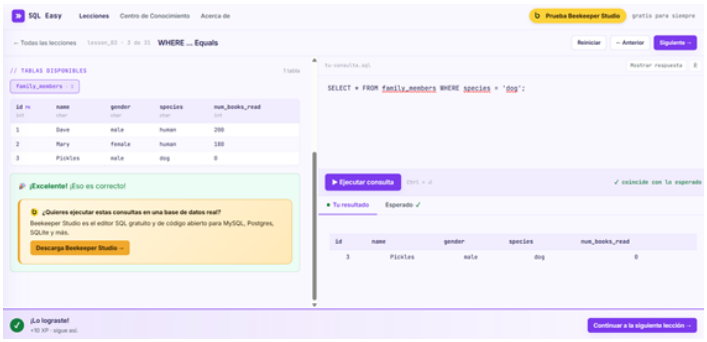
Ejercicio 1: SELECT *

Objetivo	Ver todos los datos almacenados en la tabla family_members.
Concepto SQL	SELECT con asterisco (*): devuelve todas las columnas de una tabla.
Consulta ejecutada	`SELECT * FROM family_members;`
Resultado	 *Captura del ejercicio 1 en SQL Easy*
Interpretación	Devuelve las 3 filas de la tabla (Dave, Mary y Pickles) con todas sus columnas: id, name, gender, species y num_books_read.
Aprendizaje	Comprendí la estructura básica de una consulta: SELECT indica qué traer y FROM indica de qué tabla. El asterisco es útil para explorar, aunque conviene evitarlo cuando solo se necesitan algunos campos.
Aplicación en el mundo real	Un analista que recibe una tabla nueva de clientes puede usarla para revisar rápidamente qué columnas y qué tipo de datos contiene antes de profundizar.

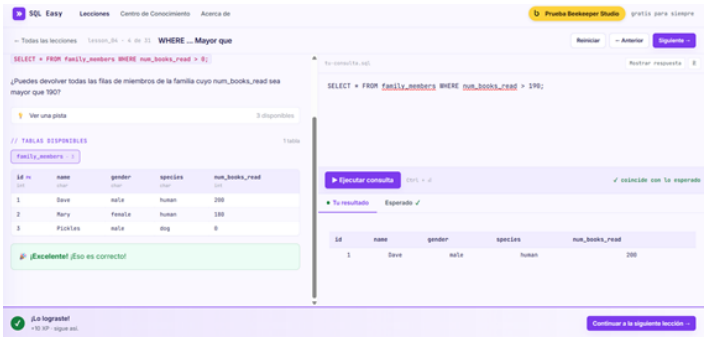
Ejercicio 2: SELECT columnas específicas

Objetivo	Obtener únicamente el nombre y la especie de cada miembro de la familia.
Concepto SQL	SELECT con lista de columnas separadas por coma.
Consulta ejecutada	`SELECT name, species FROM family_members;`
Resultado	 *Captura del ejercicio 2 en SQL Easy*
Interpretación	Muestra solo las columnas name y species de los 3 registros: Dave (human), Mary (human) y Pickles (dog).
Aprendizaje	Aprendí que se pueden elegir las columnas exactas separándolas con comas. Esto reduce el volumen de datos y hace los resultados más legibles.
Aplicación en el mundo real	Generar una lista de contactos con solo nombre y correo, sin exponer datos sensibles como dirección o documento de identidad.

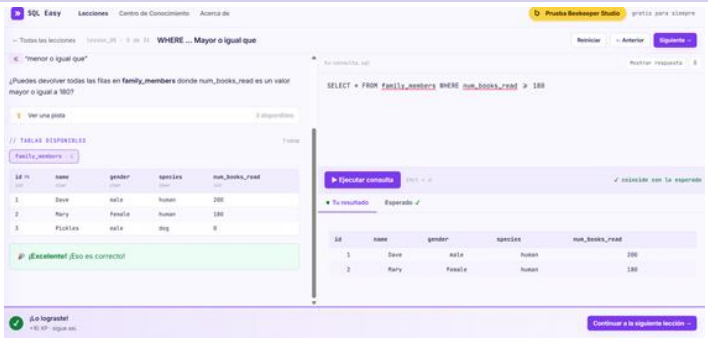
Ejercicio 3: WHERE ... Equals

Objetivo	Filtrar los miembros de la familia que sean perros.
Concepto SQL	Cláusula WHERE con el operador de igualdad (=).
Consulta ejecutada	`SELECT * FROM family_members WHERE species = 'dog';`
Resultado	 *Captura del ejercicio 3 en SQL Easy*
Interpretación	Devuelve una sola fila: Pickles (id 3, male, dog, 0 libros leídos), porque es el único registro cuya especie es 'dog'.
Aprendizaje	Entendí que WHERE filtra filas antes de mostrarlas y que los valores de texto se escriben entre comillas simples.
Aplicación en el mundo real	Filtrar los pedidos de un cliente específico o los empleados de un área determinada en una base de datos corporativa.

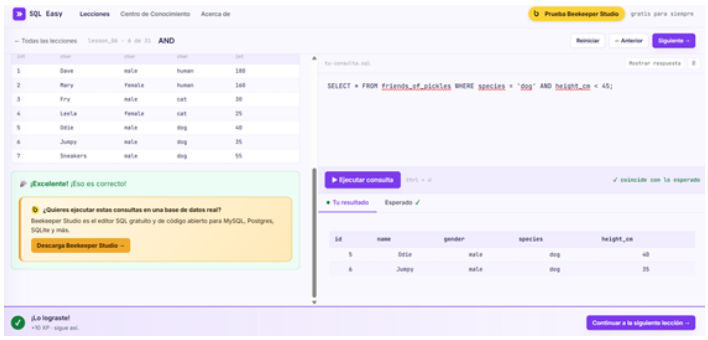
Ejercicio 4: WHERE ... Mayor que

Objetivo	Encontrar los miembros cuyo número de libros leídos sea mayor que 190.
Concepto SQL	Operador de comparación mayor que (>) en WHERE.
Consulta ejecutada	`SELECT * FROM family_members WHERE num_books_read > 190;`
Resultado	 *Captura del ejercicio 4 en SQL Easy*
Interpretación	Retorna únicamente a Dave, que leyó 200 libros. Mary (180) y Pickles (0) quedan excluidos porque no superan 190.
Aprendizaje	Aprendí a filtrar por valores numéricos y que el operador > excluye el valor límite: solo se incluyen los estrictamente mayores.
Aplicación en el mundo real	Identificar ventas superiores a un monto objetivo o clientes con compras por encima de cierto umbral.

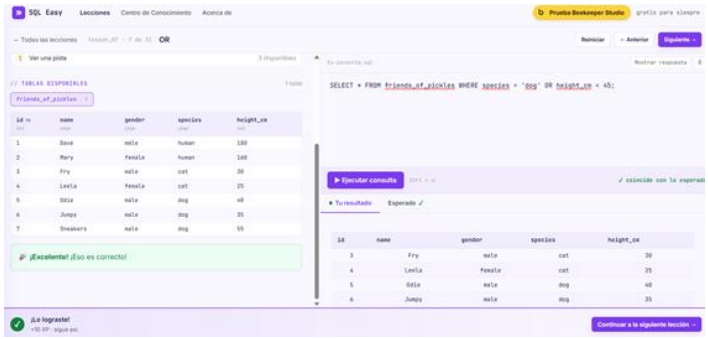
Ejercicio 5: WHERE ... Mayor o igual que

Objetivo	Listar los miembros que leyeron 180 libros o más.
Concepto SQL	Operador de comparación mayor o igual que (>=).
Consulta ejecutada	`SELECT * FROM family_members WHERE num_books_read >= 180;`
Resultado	 *Captura del ejercicio 5 en SQL Easy*
Interpretación	Devuelve a Dave (200) y Mary (180). Mary aparece porque el operador >= incluye el valor límite.
Aprendizaje	Comprendí la diferencia entre > y >=: con el segundo, el valor exacto del umbral sí se incluye en el resultado.
Aplicación en el mundo real	Seleccionar estudiantes con nota mínima aprobatoria o empleados que alcanzaron la meta de ventas.

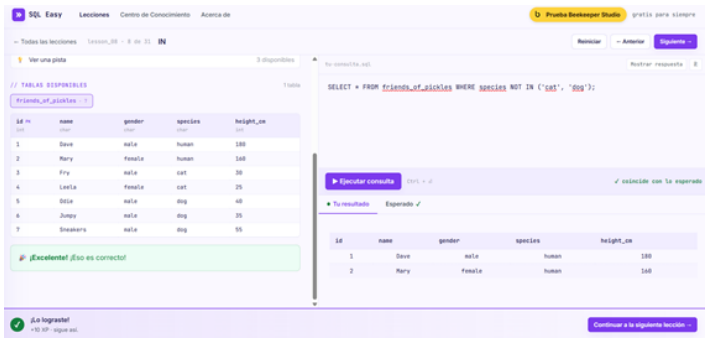
Ejercicio 6: AND

Objetivo	Encontrar los perros cuya altura sea menor a 45 cm.
Concepto SQL	Operador lógico AND: combina condiciones que deben cumplirse todas.
Consulta ejecutada	`SELECT * FROM friends_of_pickles WHERE species = 'dog' AND height_cm < 45;`
Resultado	 *Captura del ejercicio 6 en SQL Easy*
Interpretación	Devuelve Odie (40 cm) y Jumpy (35 cm), los dos perros que cumplen ambas condiciones a la vez. Sneakers (55 cm) es perro pero no cumple la altura.
Aprendizaje	Aprendí que con AND una fila solo aparece si todas las condiciones son verdaderas, lo que permite búsquedas más precisas.
Aplicación en el mundo real	Buscar productos de una categoría específica cuyo stock sea menor a un mínimo, para decidir reposiciones.

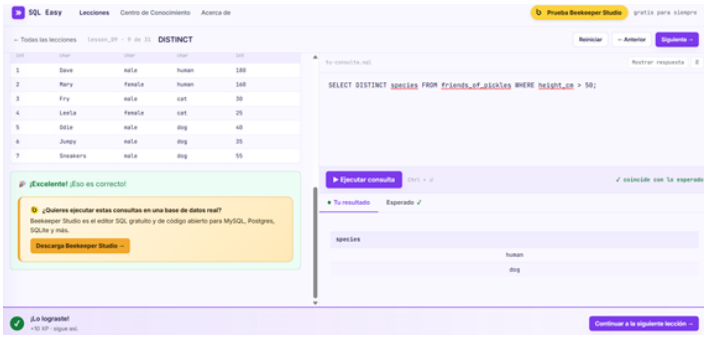
Ejercicio 7: OR

Objetivo	Obtener los registros que sean perros o que midan menos de 45 cm.
Concepto SQL	Operador lógico OR: basta con que se cumpla una de las condiciones.
Consulta ejecutada	`SELECT * FROM friends_of_pickles WHERE species = 'dog' OR height_cm < 45;`
Resultado	 *Captura del ejercicio 7 en SQL Easy*
Interpretación	Incluye a Fry y Leela (gatos de menos de 45 cm) además de Odie, Jumpy y Sneakers (perros). Aparecen todos los que cumplen al menos una condición.
Aprendizaje	Entendí el contraste con AND: OR amplía el resultado porque cada fila solo necesita cumplir una de las condiciones.
Aplicación en el mundo real	Listar clientes que sean de Lima o que hayan comprado más de cierto monto para una campaña comercial.

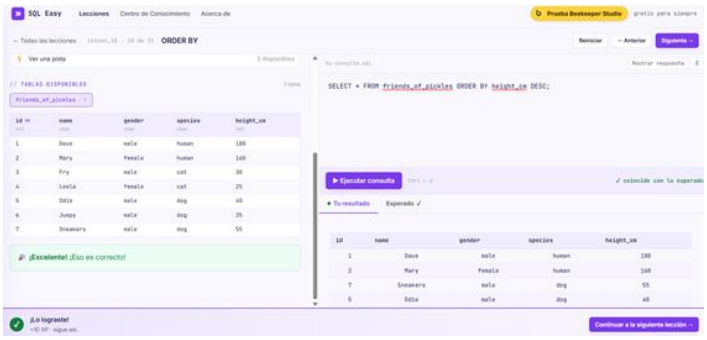
Ejercicio 8: IN

Objetivo	Excluir a los gatos y perros para ver los demás registros.
Concepto SQL	Operador IN / NOT IN: compara un valor contra una lista de opciones.
Consulta ejecutada	`SELECT * FROM friends_of_pickles WHERE species NOT IN ('cat', 'dog');`
Resultado	 *Captura del ejercicio 8 en SQL Easy*
Interpretación	Devuelve solo a Dave y Mary, ambos de especie 'human', porque ninguna de las otras filas pertenece a 'cat' o 'dog'.
Aprendizaje	Aprendí que IN reemplaza varias condiciones OR encadenadas y que NOT IN invierte la lógica, haciendo la consulta más corta y legible.
Aplicación en el mundo real	Excluir de un reporte las órdenes con estados 'cancelado' o 'devuelto' sin escribir muchas condiciones.

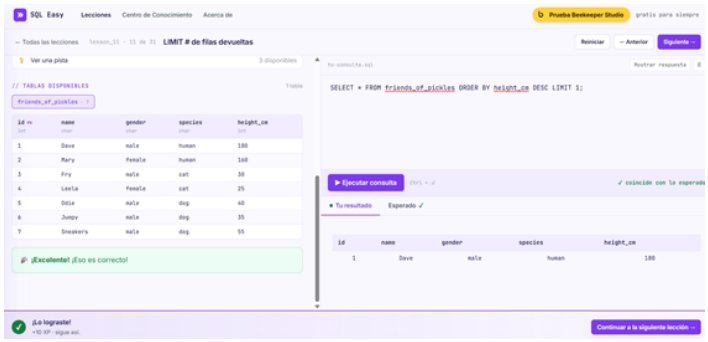
Ejercicio 9: DISTINCT

Objetivo	Conocer las especies distintas entre quienes miden más de 50 cm.
Concepto SQL	DISTINCT: elimina valores duplicados del resultado.
Consulta ejecutada	`SELECT DISTINCT species FROM friends_of_pickles WHERE height_cm > 50;`
Resultado	 *Captura del ejercicio 9 en SQL Easy*
Interpretación	Devuelve dos valores únicos: human y dog. Aunque hay varias filas que cumplen la condición, cada especie aparece una sola vez.
Aprendizaje	Comprendí que DISTINCT sirve para obtener listas de valores únicos y evitar repeticiones en los resultados.
Aplicación en el mundo real	Obtener la lista de ciudades o categorías de productos existentes sin repetir cada una por cada registro.

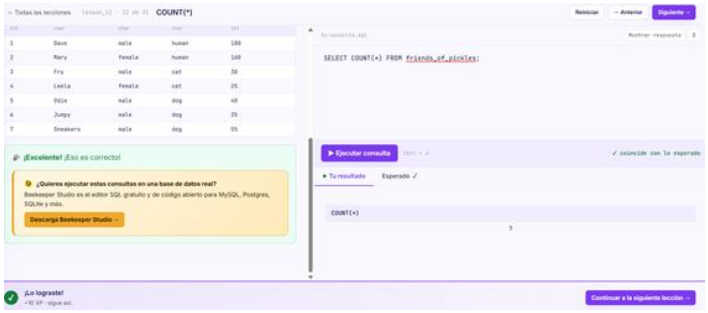
Ejercicio 10: ORDER BY

Objetivo	Ordenar los registros por altura de mayor a menor.
Concepto SQL	ORDER BY con DESC: ordena el resultado por una columna.
Consulta ejecutada	`SELECT * FROM friends_of_pickles ORDER BY height_cm DESC;`
Resultado	 *Captura del ejercicio 10 en SQL Easy*
Interpretación	La tabla se devuelve ordenada desde Dave (180 cm), luego Mary (160), Sneakers (55), Odie (40), y así hasta el más bajo.
Aprendizaje	Aprendí que el orden por defecto es ascendente y que DESC lo invierte. El orden es clave para leer rankings.
Aplicación en el mundo real	Crear rankings de ventas, productos más vendidos o empleados con mayor desempeño.

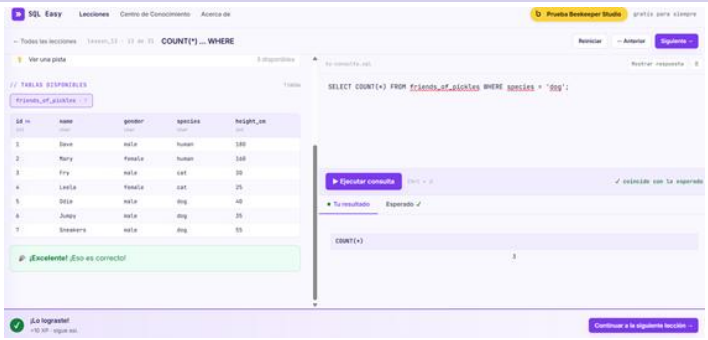
Ejercicio 11: LIMIT # de filas devueltas

Objetivo	Obtener solo el registro con la mayor altura.
Concepto SQL	LIMIT: restringe la cantidad de filas del resultado.
Consulta ejecutada	`SELECT * FROM friends_of_pickles ORDER BY height_cm DESC LIMIT 1;`
Resultado	 *Captura del ejercicio 11 en SQL Easy*
Interpretación	Devuelve únicamente una fila: Dave, con 180 cm, la altura más alta de la tabla.
Aprendizaje	Entendí que combinar ORDER BY con LIMIT es una forma práctica de obtener el 'top N' de cualquier criterio.
Aplicación en el mundo real	Mostrar los 10 clientes con más compras o los 5 productos más caros en un panel de control.

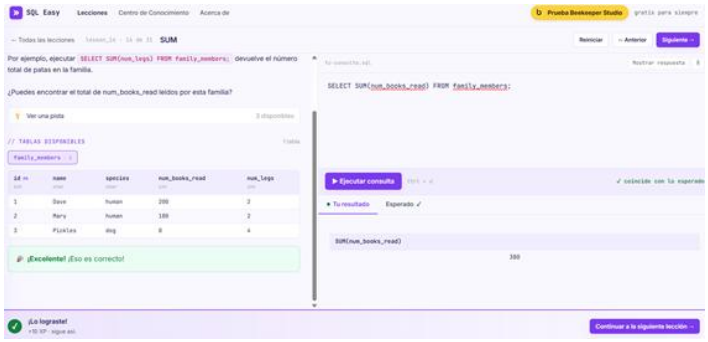
Ejercicio 12: COUNT(*)

Objetivo	Saber cuántos registros tiene la tabla friends_of_pickles.
Concepto SQL	COUNT(*): función de agregación que cuenta filas.
Consulta ejecutada	`SELECT COUNT(*) FROM friends_of_pickles;`
Resultado	 *Captura del ejercicio 12 en SQL Easy*
Interpretación	El resultado es 7, que corresponde al número total de filas de la tabla.
Aprendizaje	Aprendí que las funciones de agregación resumen muchas filas en un solo valor, en lugar de listar los datos.
Aplicación en el mundo real	Contar el total de clientes registrados, pedidos del mes o incidencias abiertas.

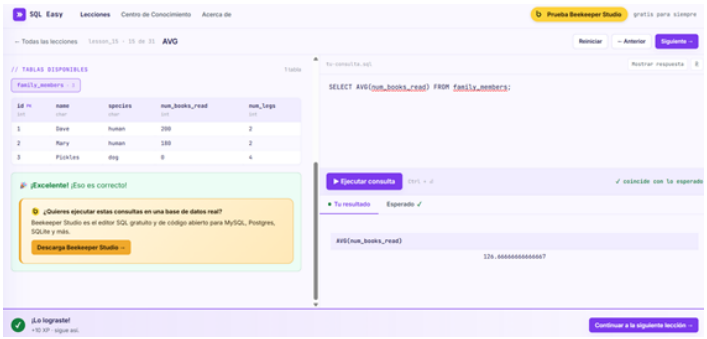
Ejercicio 13: COUNT(*) ... WHERE

Objetivo	Contar cuántos perros hay en la tabla.
Concepto SQL	COUNT(*) combinado con WHERE para contar solo filas filtradas.
Consulta ejecutada	`SELECT COUNT(*) FROM friends_of_pickles WHERE species = 'dog';`
Resultado	 *Captura del ejercicio 13 en SQL Easy*
Interpretación	Devuelve 3, es decir, Odie, Jumpy y Sneakers.
Aprendizaje	Comprendí que WHERE se aplica antes del conteo, por lo que COUNT solo considera las filas que pasan el filtro.
Aplicación en el mundo real	Calcular cuántos pedidos están pendientes o cuántos empleados pertenecen a un departamento.

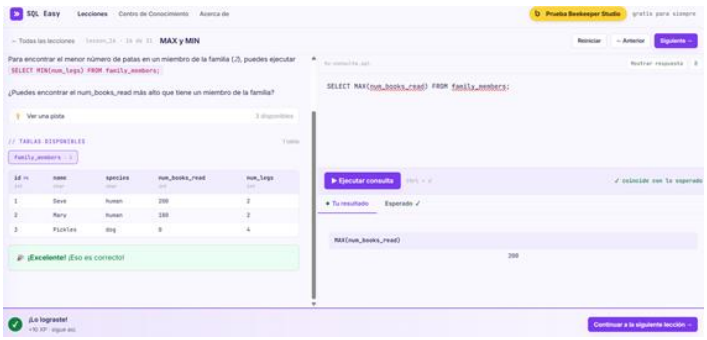
Ejercicio 14: SUM

Objetivo	Calcular el total de libros leídos por la familia.
Concepto SQL	SUM: suma los valores de una columna numérica.
Consulta ejecutada	`SELECT SUM(num_books_read) FROM family_members;`
Resultado	 *Captura del ejercicio 14 en SQL Easy*
Interpretación	El resultado es 380, que equivale a 200 (Dave) + 180 (Mary) + 0 (Pickles).
Aprendizaje	Aprendí a totalizar una columna numérica con una sola instrucción, sin sumar manualmente.
Aplicación en el mundo real	Obtener el total de ingresos de un periodo o la suma de horas trabajadas por un equipo.

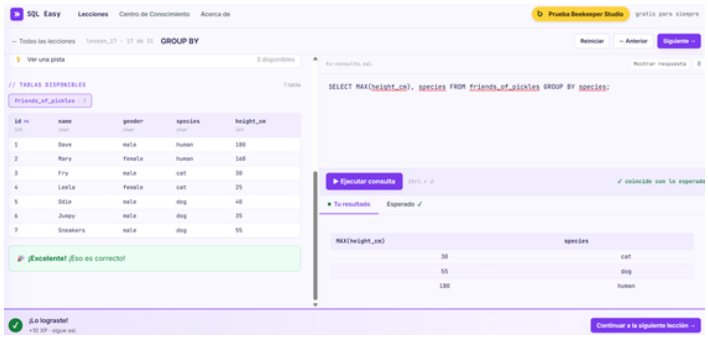
Ejercicio 15: AVG

Objetivo	Calcular el promedio de libros leídos por miembro de la familia.
Concepto SQL	AVG: calcula el promedio de una columna numérica.
Consulta ejecutada	`SELECT AVG(num_books_read) FROM family_members;`
Resultado	 *Captura del ejercicio 15 en SQL Easy*
Interpretación	Devuelve 126.666..., resultado de dividir 380 entre los 3 miembros.
Aprendizaje	Entendí que AVG divide la suma entre la cantidad de filas con valor, y que el resultado puede tener muchos decimales.
Aplicación en el mundo real	Calcular el ticket promedio de compra, la nota promedio de un curso o el salario promedio por área.

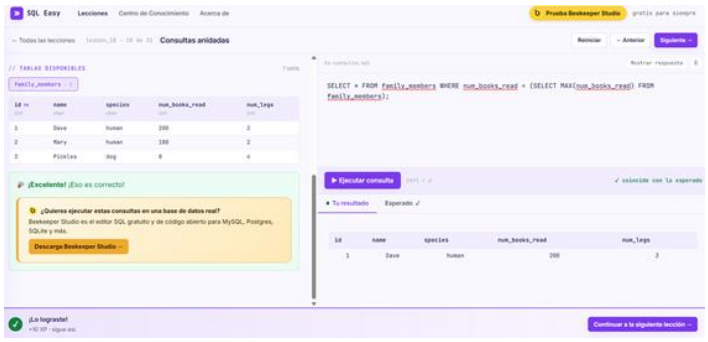
Ejercicio 16: MAX y MIN

Objetivo	Hallar el mayor número de libros leídos por un miembro de la familia.
Concepto SQL	MAX (y MIN): devuelven el valor máximo (o mínimo) de una columna.
Consulta ejecutada	`SELECT MAX(num_books_read) FROM family_members;`
Resultado	 *Captura del ejercicio 16 en SQL Easy*
Interpretación	El resultado es 200, el valor más alto de la columna num_books_read (le corresponde a Dave).
Aprendizaje	Aprendí que MAX y MIN ubican valores extremos sin necesidad de ordenar toda la tabla.
Aplicación en el mundo real	Identificar la venta más alta, el precio mínimo de un producto o la fecha más reciente de un registro.

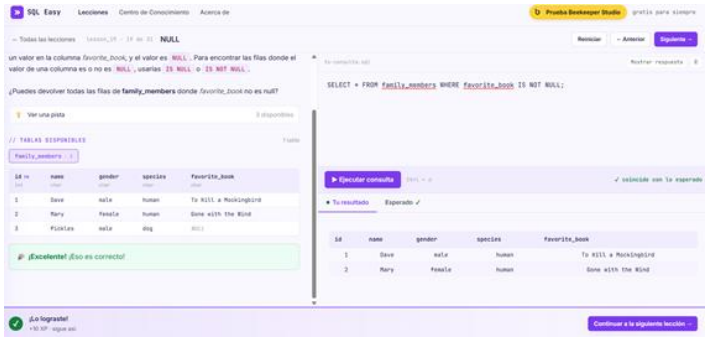
Ejercicio 17: GROUP BY

Objetivo	Obtener la altura máxima de cada especie.
Concepto SQL	GROUP BY: agrupa filas para aplicar funciones de agregación por categoría.
Consulta ejecutada	`SELECT MAX(height_cm), species FROM friends_of_pickles GROUP BY species;`
Resultado	 *Captura del ejercicio 17 en SQL Easy*
Interpretación	Devuelve una fila por especie: cat con 30 cm, dog con 55 cm y human con 180 cm.
Aprendizaje	Comprendí que GROUP BY divide los datos en grupos y que la función de agregación se calcula por cada grupo.
Aplicación en el mundo real	Calcular ventas totales por región, o cantidad de empleados por departamento.

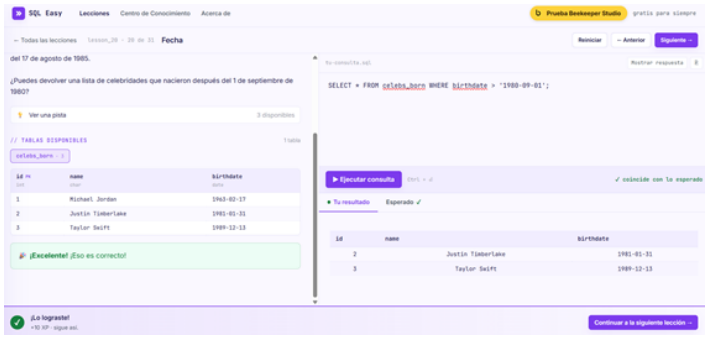
Ejercicio 18: Consultas anidadas

Objetivo	Encontrar al miembro que leyó el mayor número de libros, sin conocer el valor de antemano.
Concepto SQL	Subconsulta: una consulta dentro de otra, usada en la condición WHERE.
Consulta ejecutada	`SELECT * FROM family_members` `WHERE num_books_read = (SELECT MAX(num_books_read) FROM family_members);`
Resultado	 *Captura del ejercicio 18 en SQL Easy*
Interpretación	La subconsulta calcula el máximo (200) y la consulta externa devuelve la fila que coincide con ese valor: Dave.
Aprendizaje	Aprendí que el resultado de una consulta puede alimentar a otra, lo que permite resolver preguntas en dos pasos dentro de una sola instrucción.
Aplicación en el mundo real	Buscar al cliente con la compra más alta o a los empleados que ganan más que el promedio de la empresa.

Ejercicio 19: NULL

Objetivo	Listar los miembros que sí tienen un libro favorito registrado.
Concepto SQL	NULL: valor ausente; se evalúa con IS NULL / IS NOT NULL.
Consulta ejecutada	`SELECT * FROM family_members WHERE favorite_book IS NOT NULL;`
Resultado	 *Captura del ejercicio 19 en SQL Easy*
Interpretación	Devuelve a Dave (To Kill a Mockingbird) y Mary (Gone with the Wind). Pickles se excluye porque su favorite_book es NULL.
Aprendizaje	Entendí que NULL no es igual a cero ni a texto vacío, y que no se compara con '=', sino con IS NULL o IS NOT NULL.
Aplicación en el mundo real	Detectar clientes sin correo electrónico o registros con datos incompletos para depuración.

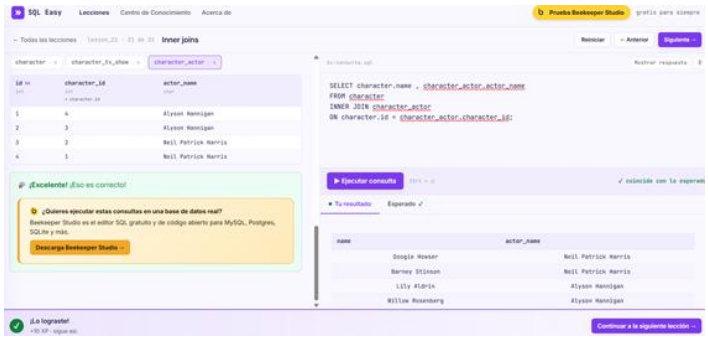
Ejercicio 20: Fecha

Objetivo	Listar las celebridades nacidas después del 1 de septiembre de 1980.
Concepto SQL	Comparación de fechas con el formato AAAA-MM-DD.
Consulta ejecutada	`SELECT * FROM celebs_born WHERE birthdate > '1980-09-01';`
Resultado	 *Captura del ejercicio 20 en SQL Easy*
Interpretación	Devuelve a Justin Timberlake (1981-01-31) y Taylor Swift (1989-12-13). Michael Jordan (1963-02-17) queda fuera.
Aprendizaje	Aprendí que las fechas se escriben como texto en formato ISO y se pueden comparar con los operadores habituales.
Aplicación en el mundo real	Obtener pedidos posteriores a una fecha, contratos vigentes o clientes registrados en el último año.

Ejercicio 21: Inner joins

Objetivo	Mostrar cada personaje junto al nombre de su actor.
Concepto SQL	INNER JOIN: une dos tablas y devuelve solo las filas con coincidencia.

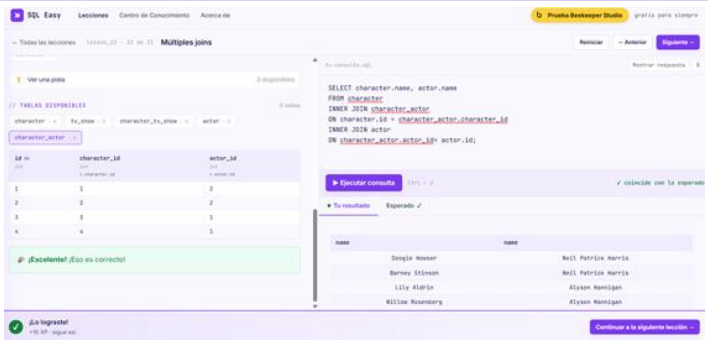
Consulta ejecutada	`SELECT character.name, character_actor.actor_name`
	`FROM character`
	`INNER JOIN character_actor`
	`ON character.id = character_actor.character_id;`

Resultado	 *Captura del ejercicio 21 en SQL Easy*
Interpretación	Cada fila combina un personaje con su actor, por ejemplo Doogie Howser con Neil Patrick Harris y Lily Aldrin con Alyson Hannigan.
Aprendizaje	Comprendí que un JOIN relaciona tablas mediante una clave (id y character_id) y que INNER solo conserva coincidencias.
Aplicación en el mundo real	Unir la tabla de pedidos con la de clientes para ver el nombre del cliente en cada pedido.

Ejercicio 22: Múltiples joins

Objetivo	Obtener personajes y actores a través de una tabla intermedia.
Concepto SQL	Varios JOIN encadenados para relacionar más de dos tablas.

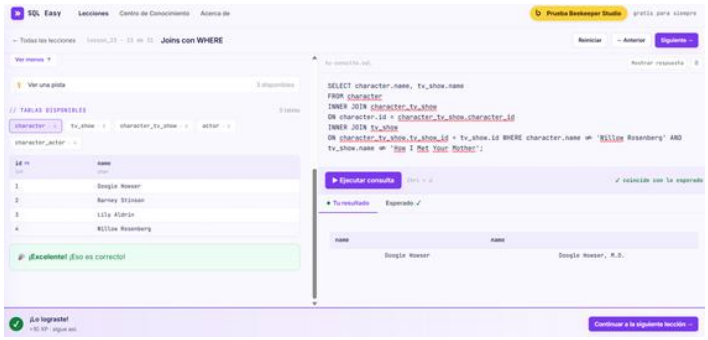
Consulta ejecutada	`SELECT character.name, actor.name`
	`FROM character`
	`INNER JOIN character_actor`
	`ON character.id = character_actor.character_id`
	`INNER JOIN actor`
	`ON character_actor.actor_id = actor.id;`

Resultado	 *Captura del ejercicio 22 en SQL Easy*
Interpretación	Une tres tablas (character, character_actor y actor) y devuelve el nombre del personaje con el nombre del actor, por ejemplo Barney Stinson con Neil Patrick Harris.
Aprendizaje	Aprendí que una tabla intermedia resuelve relaciones de muchos a muchos y que los JOIN se pueden encadenar uno tras otro.
Aplicación en el mundo real	Relacionar estudiantes, matrículas y cursos, o productos, órdenes y clientes en un sistema de ventas.

Ejercicio 23: Joins con WHERE

Objetivo	Filtrar el resultado de un JOIN para un personaje y una serie concretos.
Concepto SQL	JOIN combinado con WHERE para filtrar filas ya unidas.

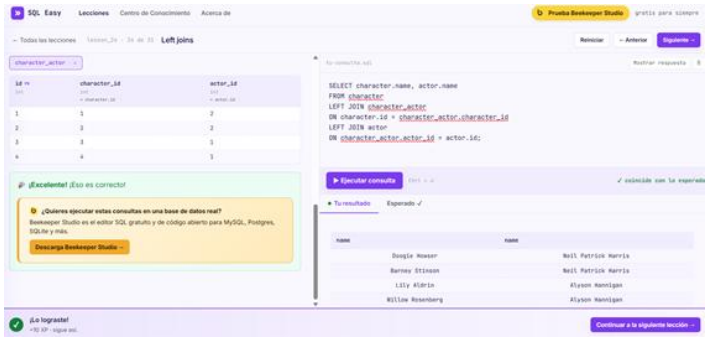
Consulta ejecutada	`SELECT character.name, tv_show.name`
	`FROM character`
	`INNER JOIN character_tv_show`
	`ON character.id = character_tv_show.character_id`
	`INNER JOIN tv_show`
	`ON character_tv_show.tv_show_id = tv_show.id`
	`WHERE character.name <> 'Willow Rosenberg'`
	`AND tv_show.name <> 'How I Met Your Mother';`

Resultado	 *Captura del ejercicio 23 en SQL Easy*
Interpretación	Devuelve los personajes y series que quedan después de excluir a Willow Rosenberg y la serie How I Met Your Mother; en la captura aparece Doogie Howser con Doogie Howser, M.D.
Aprendizaje	Entendí que WHERE se aplica sobre el resultado de las tablas unidas y que se pueden combinar varias condiciones con AND.
Aplicación en el mundo real	Consultar los pedidos de un cliente específico dentro de un rango de fechas uniendo varias tablas.

Ejercicio 24: Left joins

Objetivo	Mostrar todos los personajes, incluso los que no tengan actor asociado.
Concepto SQL	LEFT JOIN: conserva todas las filas de la tabla izquierda.

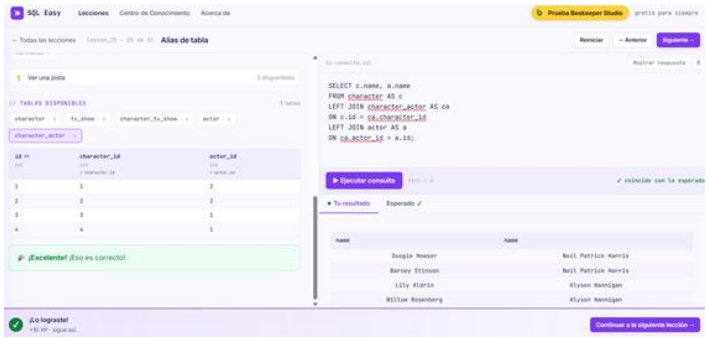
Consulta ejecutada	`SELECT character.name, actor.name`
	`FROM character`
	`LEFT JOIN character_actor`
	`ON character.id = character_actor.character_id`
	`LEFT JOIN actor`
	`ON character_actor.actor_id = actor.id;`

Resultado	 *Captura del ejercicio 24 en SQL Easy*
Interpretación	Devuelve todos los personajes con su actor; si algún personaje no tuviera actor, aparecería igual con valor NULL en esa columna.
Aprendizaje	Aprendí la diferencia con INNER JOIN: LEFT JOIN no descarta filas de la tabla principal aunque no tengan coincidencia.
Aplicación en el mundo real	Listar todos los clientes, incluso los que aún no han realizado ninguna compra.

Ejercicio 25: Alias de tabla

Objetivo	Reescribir el LEFT JOIN usando nombres cortos para las tablas.
Concepto SQL	Alias de tabla con AS.

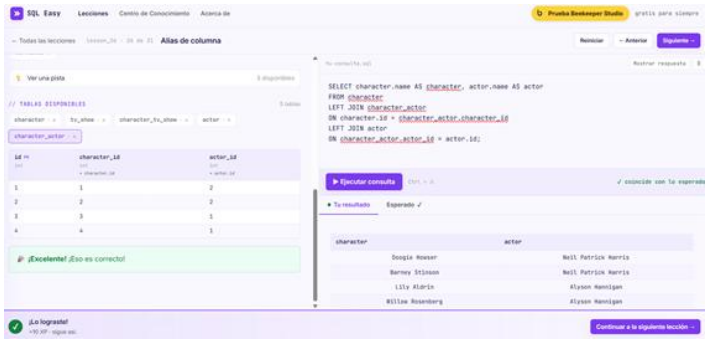
Consulta ejecutada	`SELECT c.name, a.name`
	`FROM character AS c`
	`LEFT JOIN character_actor AS ca`
	`ON c.id = ca.character_id`
	`LEFT JOIN actor AS a`
	`ON ca.actor_id = a.id;`

Resultado	 *Captura del ejercicio 25 en SQL Easy*
Interpretación	Devuelve el mismo resultado que la consulta anterior, pero con una sintaxis más corta gracias a los alias c, ca y a.
Aprendizaje	Comprendí que los alias hacen más legibles las consultas largas y evitan escribir el nombre completo de la tabla en cada columna.
Aplicación en el mundo real	Mantener consultas extensas ordenadas y fáciles de revisar por otros miembros del equipo.

Ejercicio 26: Alias de columna

Objetivo	Renombrar las columnas del resultado como 'character' y 'actor'.
Concepto SQL	Alias de columna con AS.

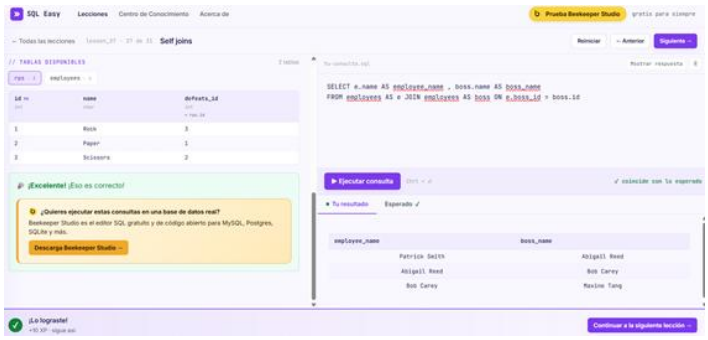
Consulta ejecutada	`SELECT character.name AS character, actor.name AS actor`
	`FROM character`
	`LEFT JOIN character_actor`
	`ON character.id = character_actor.character_id`
	`LEFT JOIN actor`
	`ON character_actor.actor_id = actor.id;`

Resultado	 *Captura del ejercicio 26 en SQL Easy*
Interpretación	Las columnas del resultado ahora se llaman character y actor, en lugar de mostrar dos columnas con el mismo nombre 'name'.
Aprendizaje	Aprendí que un alias cambia solo el nombre mostrado, no el dato, y elimina ambigüedades cuando dos columnas se llaman igual.
Aplicación en el mundo real	Preparar reportes con encabezados claros, como 'Ventas totales' o 'Nombre del cliente', para quienes los leen.

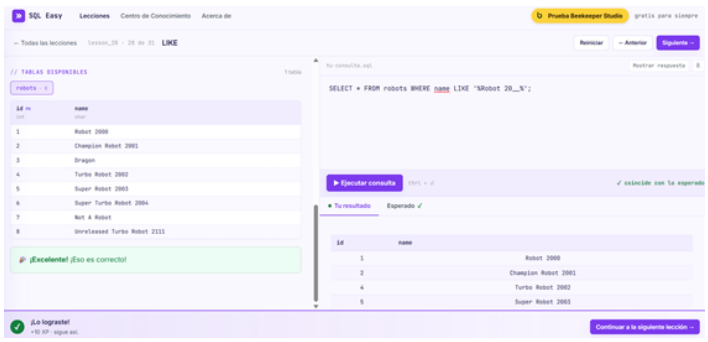
Ejercicio 27: Self joins

Objetivo	Mostrar cada empleado junto al nombre de su jefe.
Concepto SQL	Self join: unir una tabla consigo misma usando alias distintos.

Consulta ejecutada	`SELECT e.name AS employee_name, boss.name AS boss_name`
	`FROM employees AS e`
	`JOIN employees AS boss`
	`ON e.boss_id = boss.id;`

Resultado	 *Captura del ejercicio 27 en SQL Easy*
Interpretación	Cada fila muestra a un empleado y su jefe: Patrick Smith con Abigail Reed, Abigail Reed con Bob Carey y Bob Carey con Maxine Tang.
Aprendizaje	Entendí que una misma tabla puede jugar dos roles y que los alias permiten distinguir al empleado del jefe.
Aplicación en el mundo real	Representar jerarquías organizacionales, como quién reporta a quién dentro de una empresa.

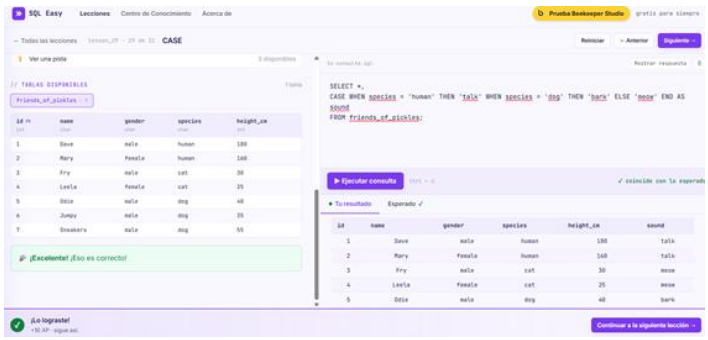
Ejercicio 28: LIKE

Objetivo	Encontrar los robots cuyo nombre contenga un patrón del tipo 'Robot 20__'.
Concepto SQL	LIKE con comodines: % (cualquier cantidad de caracteres) y _ (un carácter).
Consulta ejecutada	`SELECT * FROM robots WHERE name LIKE '%Robot 20__%';`
Resultado	 *Captura del ejercicio 28 en SQL Easy*
Interpretación	Devuelve robots como Robot 2000, Champion Robot 2001, Turbo Robot 2002 y Super Robot 2003, cuyos nombres incluyen 'Robot 20' seguido de dos caracteres.
Aprendizaje	Aprendí que % reemplaza cualquier texto y _ reemplaza exactamente un carácter, lo que permite búsquedas por patrones.
Aplicación en el mundo real	Buscar clientes por apellido parcial, correos de un dominio o códigos de producto con cierto formato.

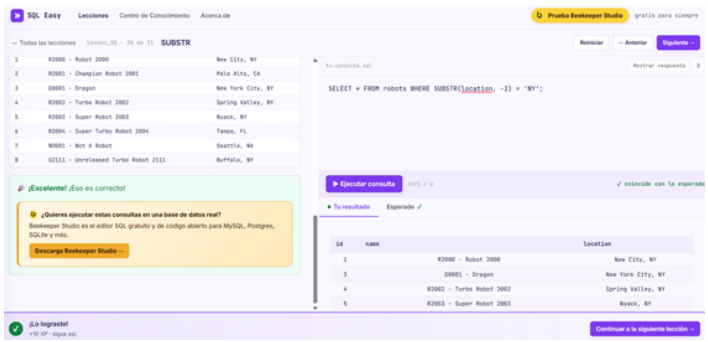
Ejercicio 29: CASE

Objetivo	Asignar un sonido a cada registro según su especie.
Concepto SQL	CASE WHEN: lógica condicional dentro de una consulta.

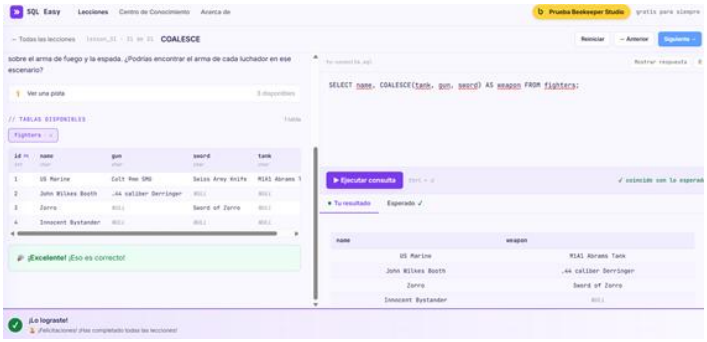
Consulta ejecutada	`SELECT *,`
	`CASE WHEN species = 'human' THEN 'talk'`
	`WHEN species = 'dog' THEN 'bark'`
	`ELSE 'meow' END AS sound`
	`FROM friends_of_pickles;`

Resultado	 *Captura del ejercicio 29 en SQL Easy*
Interpretación	Agrega la columna sound: 'talk' para humanos, 'bark' para perros y 'meow' para los demás (gatos).
Aprendizaje	Comprendí que CASE funciona como un if/else dentro de SQL y que ELSE cubre todos los casos no contemplados.
Aplicación en el mundo real	Clasificar clientes en niveles (oro, plata, bronce) según su monto de compras o etiquetar notas como aprobado/desaprobado.

Ejercicio 30: SUBSTR

Objetivo	Encontrar los robots ubicados en Nueva York usando los últimos dos caracteres de la ubicación.
Concepto SQL	SUBSTR: extrae una parte de un texto; con posición negativa cuenta desde el final.
Consulta ejecutada	`SELECT * FROM robots WHERE SUBSTR(location, -2) = 'NY';`
Resultado	 *Captura del ejercicio 30 en SQL Easy*
Interpretación	Devuelve los robots cuya ubicación termina en 'NY', como Robot 2000 (New York City, NY), Dragon (New York City, NY), Turbo Robot 2002 (Spring Valley, NY) y Super Robot 2003 (Nyack, NY).
Aprendizaje	Aprendí que SUBSTR permite trabajar con fragmentos de texto y que un valor negativo toma los caracteres desde el final.
Aplicación en el mundo real	Extraer prefijos de códigos, el año de una cadena de fecha o el código de estado/departamento de una dirección.

Ejercicio 31: COALESCE

Objetivo	Mostrar el arma de cada luchador, sea de fuego, espada o tanque.
Concepto SQL	COALESCE: devuelve el primer valor no nulo de una lista.
Consulta ejecutada	`SELECT name, COALESCE(tank, gun, sword) AS weapon FROM fighters;`
Resultado	 *Captura del ejercicio 31 en SQL Easy*
Interpretación	Para cada luchador devuelve su primer arma disponible: US Marine con M1A1 Abrams Tank, John Wilkes Booth con .44 caliber Derringer y Zorro con Sword of Zorro. Innocent Bystander no tiene ninguna, por lo que queda en NULL.
Aprendizaje	Comprendí que COALESCE sirve para elegir un valor alternativo cuando hay NULL y para unificar varias columnas en una sola.
Aplicación en el mundo real	Mostrar un teléfono de contacto principal o, si no existe, el secundario; o reemplazar valores faltantes por un texto predeterminado.

Reflexión final

Temas más fáciles

- SELECT, SELECT de columnas específicas y WHERE con igualdad (ejercicios 1 a 3): su estructura es directa y se parece a una frase en lenguaje natural.
- Operadores de comparación y lógicos (ejercicios 4 a 8): una vez entendido WHERE, solo cambia la condición.
- Funciones de agregación (ejercicios 12 a 16): COUNT, SUM, AVG, MAX y MIN devuelven un solo valor y son fáciles de verificar.

Temas más retadores

- Los JOIN, en especial los múltiples y los self joins (ejercicios 21 a 27): exigen entender cómo se relacionan las tablas mediante claves antes de escribir la consulta.
- Las consultas anidadas (ejercicio 18): requieren pensar la solución en dos pasos.
- GROUP BY, CASE y COALESCE (ejercicios 17, 29 y 31): su lógica es menos intuitiva al inicio y exige practicar con distintos datos.

Aplicación en un contexto real

Las consultas practicadas se pueden aplicar en una organización para:

- Ventas y finanzas: totales, promedios y rankings con SUM, AVG, ORDER BY y LIMIT; ventas por región o producto con GROUP BY.
- Atención al cliente: unir clientes con pedidos mediante JOIN, detectar datos faltantes con IS NULL y buscar registros por patrón con LIKE.
- Recursos humanos: consultar jerarquías con self joins y clasificar al personal con CASE.
- Calidad de datos: eliminar duplicados con DISTINCT, completar vacíos con COALESCE y extraer códigos con SUBSTR.

Conclusión general

Con estos 31 ejercicios se recorrió el camino completo de una consulta SQL: seleccionar y filtrar datos, ordenarlos y resumirlos, relacionarlos entre varias tablas y transformarlos con funciones de texto y lógica condicional. La práctica interactiva permitió comprobar de inmediato cada resultado y entender por qué una consulta devuelve lo que devuelve.

Además de reforzar los conocimientos de SQL, la documentación de cada ejercicio con consulta, captura, interpretación y aplicación real ayudó a desarrollar habilidades de análisis y de comunicación técnica. Este informe queda como un recurso de consulta y como evidencia para un portafolio profesional.